

Project: Serverless Real-Time Chat Application using AWS WebSocket API

Services Used

- Amazon API Gateway (WebSocket API)
- AWS Lambda
- Amazon DynamoDB
- AWS IAM
- Amazon CloudWatch
- HTML
- CSS
- JavaScript

Description

This project implements a real-time chat application using AWS serverless services. Users connect to the application through a WebSocket API and exchange messages instantly without refreshing the browser. AWS Lambda handles connection, disconnection, and message events, while DynamoDB stores active WebSocket connection IDs. API Gateway delivers messages to all connected users.

Definition

A serverless real-time chat application allows multiple users to communicate instantly using persistent WebSocket connections without managing dedicated servers. AWS automatically scales the application based on the number of connected users.

Scope of the Project

- Create a WebSocket API using API Gateway
- Store active client connections in DynamoDB
- Handle user connections and disconnections
- Broadcast messages to all connected users
- Build a browser-based chat interface
- Monitor the application using CloudWatch
- Deploy without managing servers

Main Features

- Real-time messaging
- Multiple user support
- Instant message delivery
- Automatic connection management
- Serverless architecture
- Cloud-based deployment

This Project Covers

- WebSocket Communication
- API Gateway Routes
- AWS Lambda Functions
- Amazon DynamoDB
- IAM Roles and Policies
- CloudWatch Logs
- HTML, CSS, JavaScript
- Serverless Architecture

Methodology

1. Serverless Architecture (Implemented)

- Uses API Gateway WebSocket API
- AWS Lambda processes events
- DynamoDB stores active connections
- API Gateway Management API broadcasts messages
- CloudWatch stores logs
- No EC2 instances are required

2. Traditional WebSocket Server (Alternative)

- Uses Node.js with Express and Socket.IO
- Requires EC2 or another server
- Manual scaling and maintenance
- Higher operational cost

This project uses the serverless approach because it is scalable, cost-effective, and easier to maintain.

AWS Services Used in the Project

1. Amazon API Gateway

Definition: Creates and manages WebSocket connections.

Purpose: Provides real-time communication.

Role: Routes \$connect, \$disconnect, and sendMessage events.

2. AWS Lambda

Definition: Serverless compute service.

Purpose: Executes backend logic.

Role: Handles connection, disconnection, and message broadcasting.

3. Amazon DynamoDB

Definition: Fully managed NoSQL database.

Purpose: Stores active WebSocket connection IDs.

Role: Keeps track of connected users.

4. AWS IAM

Definition: Identity and Access Management service.

Purpose: Controls permissions.

Role: Allows Lambda to access DynamoDB and API Gateway.

5. Amazon CloudWatch

Definition: Monitoring and logging service.

Purpose: Stores Lambda logs.

Role: Used for debugging and monitoring.

Project Details

Frontend

- HTML
- CSS
- JavaScript

Backend

- AWS Lambda (Python)

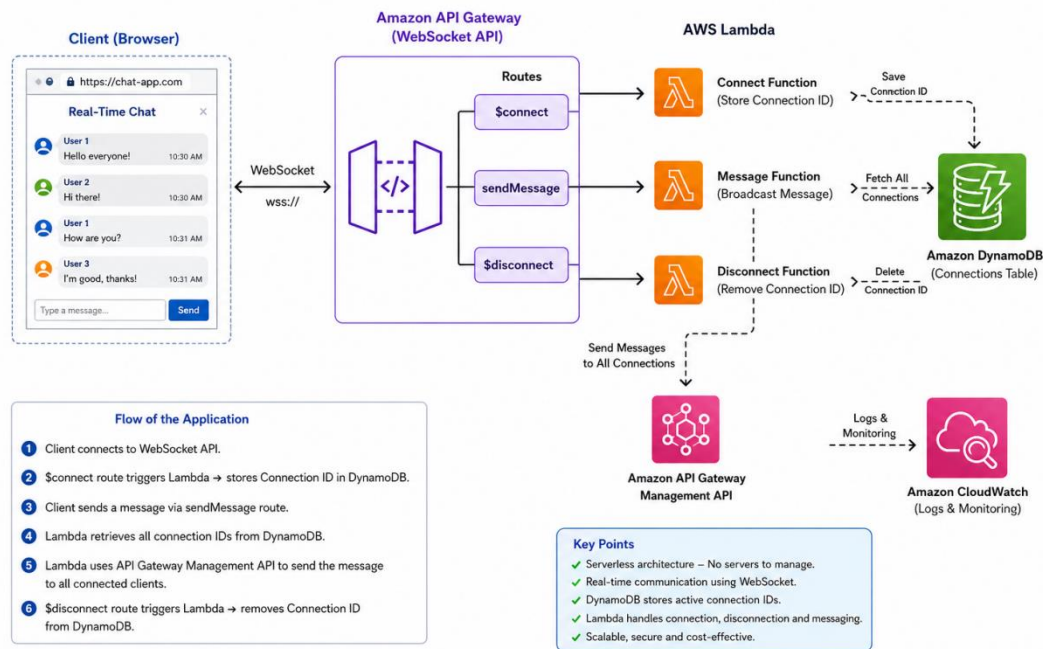
Database

- Amazon DynamoDB

Communication

- WebSocket API

Serverless Real-Time Chat Application – AWS Architecture



Implementation Steps

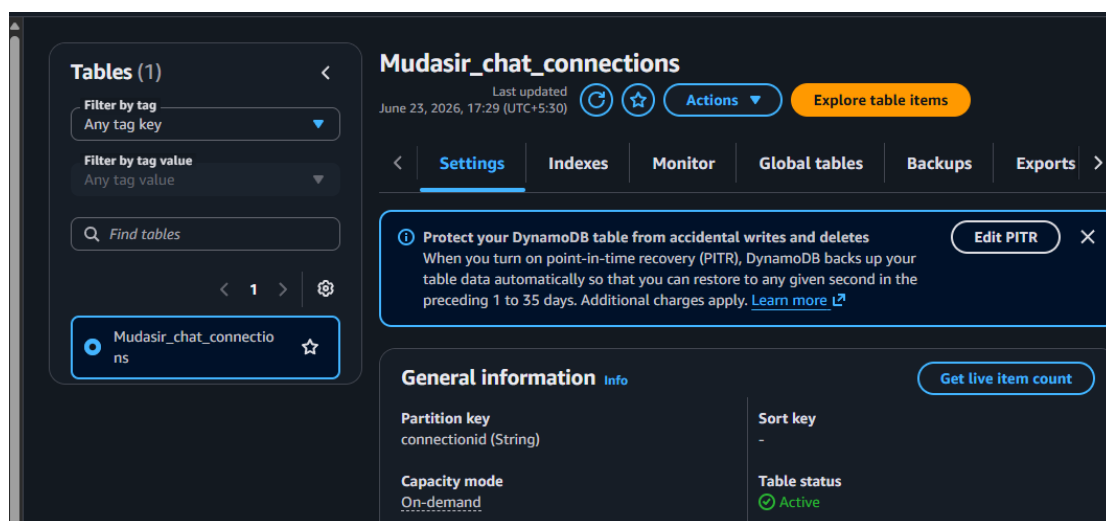
Step 1: Create DynamoDB Table

Created a DynamoDB table named **Mudasir_chat_connections**.

Partition Key:

connectionid

Purpose: Stores the WebSocket connection IDs of connected users.



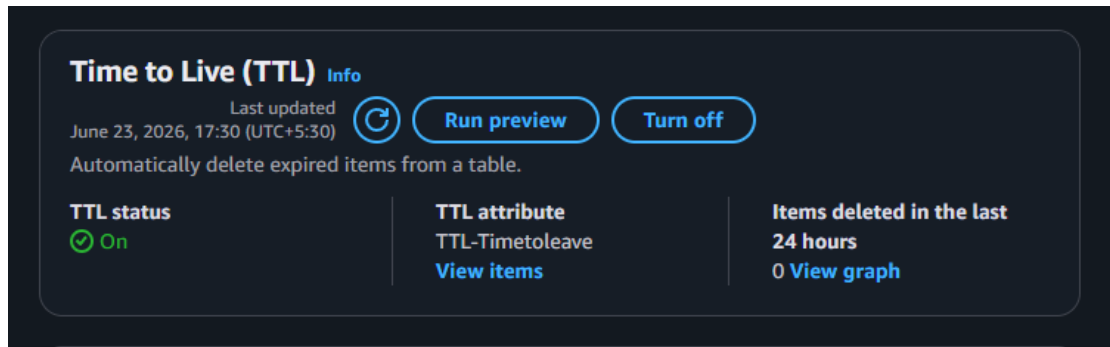
Step 2: Enable TTL

Enabled Time To Live (TTL).

TTL Attribute:

ttd

Purpose: Automatically removes inactive connection records.



Step 3: Create IAM Role

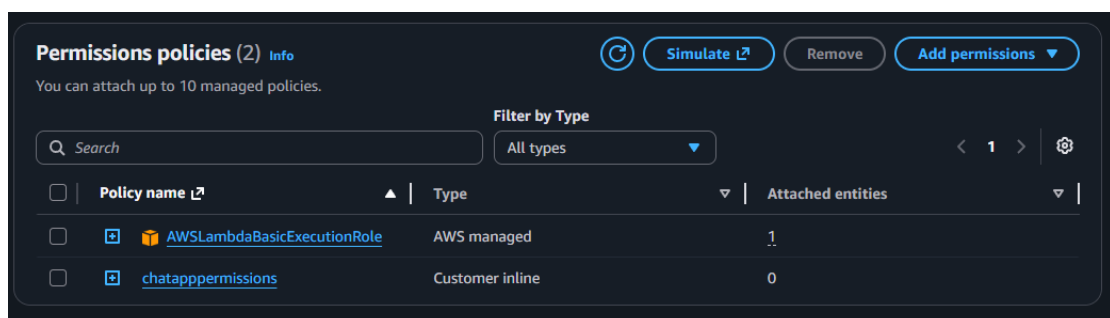
Created an IAM role for Lambda.

Role: chatapplambdarole

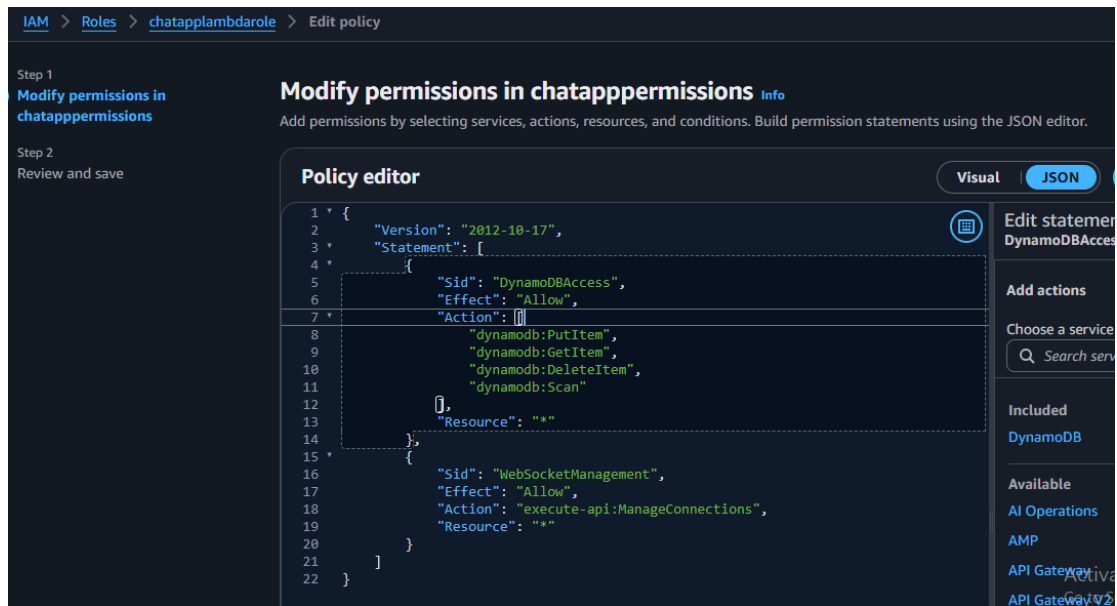
Permissions:

- DynamoDB
- CloudWatch
- execute-api:ManageConnections

Purpose: Allows Lambda to access DynamoDB and send WebSocket messages.



I have used Inline permissions (chatapppermissions):



Step 4: Create Lambda Function for [\\$connect](#)

Created the connect Lambda.

Runtime: python 3.14

Choose execution role : chatapplambdarole=

Timeout

1min3sec

Purpose: Stores the user's connection ID in DynamoDB when a client connects.'

Code:

```
import boto3
import time
from datetime import datetime, timezone

dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
table = dynamodb.Table('Mudasir_chat_connections')

def lambda_handler(event, context):
    try:
        connection_id = event['requestContext']['connectionId']

        table.put_item(Item={
            'connectionid': connection_id,
            'connectedAt': datetime.now(timezone.utc).isoformat(),
            'ttl': int(time.time()) + 7200
        })
```

```

print("Connection stored:", connection_id)

return {
    "statusCode": 200,
    "body": "Connection stored successfully"
}

except Exception as e:
    print("Error:", str(e))

    return {
        "statusCode": 500,
        "body": "Failed to store connection"
    }

```

Step 5: Create Lambda Function for `$disconnect`

Created the disconnect Lambda.

Runtime: python 3.14

Choose execution role : chatapplambdarole

Timeout

1min3sec

Purpose: Removes the connection ID from DynamoDB when a user disconnects.

Code:

```

import boto3

# Specify region explicitly — boto3 defaults to us-east-1 without this
dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
table = dynamodb.Table('Mudasir_chat_connections')

def lambda_handler(event, context):
    connection_id = event['requestContext']['connectionId']

    # $disconnect fires on clean closes (browser tab closed gracefully,
    # or client called ws.close()). For network drops or sudden tab kills,
    # $disconnect does NOT fire — TTL on the DynamoDB item is the safety net.
    table.delete_item(Key={'connectionid': connection_id})

```

```
return {'statusCode': 200}
```

Step 6: Create Lambda Function for [sendMessage](#)

Created the message Lambda.

Runtime: python 3.14

Choose execution role : chatapplambdarole

Timeout

1min3sec

Purpose: Receives chat messages and broadcasts them to all connected users.

Code:

```
import boto3
import json
import os
from datetime import datetime, timezone
from botocore.exceptions import ClientError

dynamodb = boto3.resource('dynamodb', region_name='ap-south-1')
table = dynamodb.Table('Mudasir_chat_connections')

def lambda_handler(event, context):

    if "requestContext" not in event:
        return {"statusCode": 400, "body": "Invalid event"}

    ctx = event['requestContext']

    endpoint = os.environ['WS_ENDPOINT']

    mgmt = boto3.client(
        'apigatewaymanagementapi',
        endpoint_url=endpoint,
        region_name='ap-south-1'
    )

    body = json.loads(event.get('body') or '{}')
    message = body.get('message', "")
    username = body.get('username', 'Anonymous')

    payload = json.dumps({
        "username": username,
        "message": message,
        "timestamp": datetime.now(timezone.utc).isoformat()
    })
```



```

}).encode("utf-8")

connections = table.scan(ProjectionExpression='connectionid')['Items']

for item in connections:
    try:
        mgmt.post_to_connection(
            ConnectionId=item['connectionid'],
            Data=payload
        )

    except ClientError as e:
        if e.response['Error']['Code'] == 'GoneException':
            table.delete_item(Key={'connectionid': item['connectionid']})

return {"statusCode": 200, "body": "Message broadcasted"}

```

Step 7: Create WebSocket API

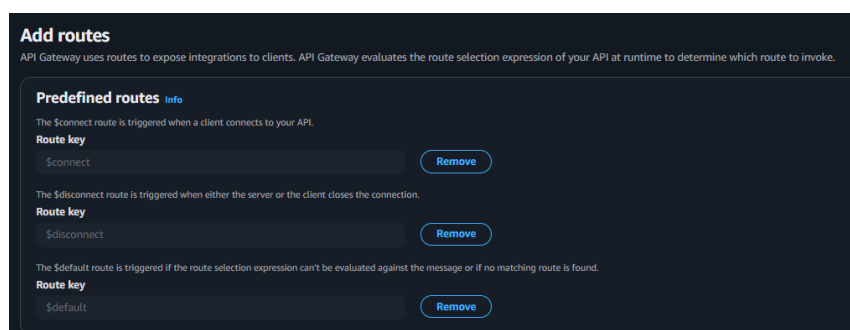
Created a WebSocket API using API Gateway.

Route selection expression: For a WebSocket API in API Gateway, Route Selection Expression determines how API Gateway chooses which route (\$connect, \$disconnect, sendMessage, etc.) to invoke.

`$request.body.action`

Routes:

- \$connect
- \$disconnect
- sendMessage



Purpose: Handles WebSocket communication.

Step 8: Integrate Lambda Functions

Connected each WebSocket route with its corresponding Lambda function.

Purpose: Processes connection, disconnection, and messaging events.

The screenshot shows three integration configurations for an API Gateway. Each configuration has an 'Integration type' dropdown set to 'Lambda'. The 'AWS Region' is set to 'ap-south-1'. The 'Lambda function' is specified as 'arn:aws:lambda:ap-south-1:610269527537:function:chatconnect' for the '\$connect' route, 'arn:aws:lambda:ap-south-1:610269527537:function:chatdisconnect' for the '\$disconnect' route, and 'arn:aws:lambda:ap-south-1:610269527537:function:chatsendmessage' for the '\$default' route.

Step 9: Deploy API

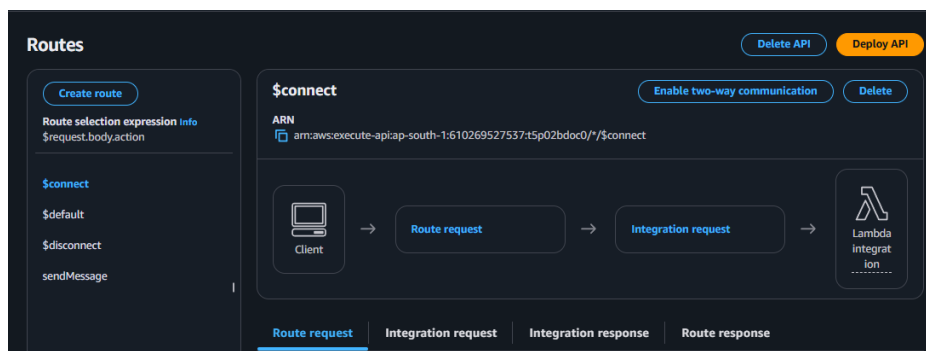
Created a deployment stage.

Stage Name:

prod1

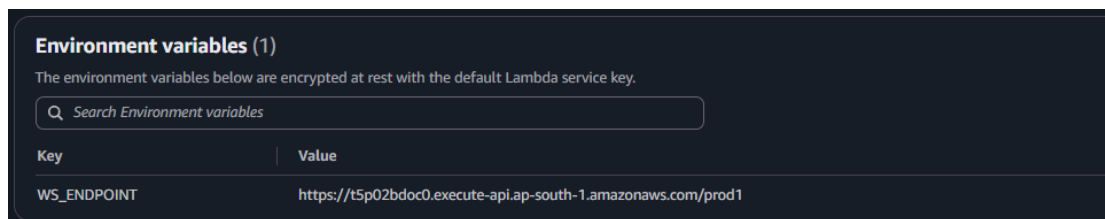
Purpose: Makes the WebSocket API publicly accessible.

The 'Add stages' dialog shows a 'Stage name' input field containing 'prod1'. There are 'Add stage' and 'Remove' buttons.



Step 10: Configure Environment Variable

Added the environment variable: in chatsendmessage lambda.



Purpose: Allows Lambda to send messages back through the WebSocket API.

Step 11: Create Chat Interface

Developed a simple frontend using HTML, CSS, and JavaScript.

Features:

- Connect button
- Chat window
- Message input
- Send button

Purpose: Provides a user interface for chatting.

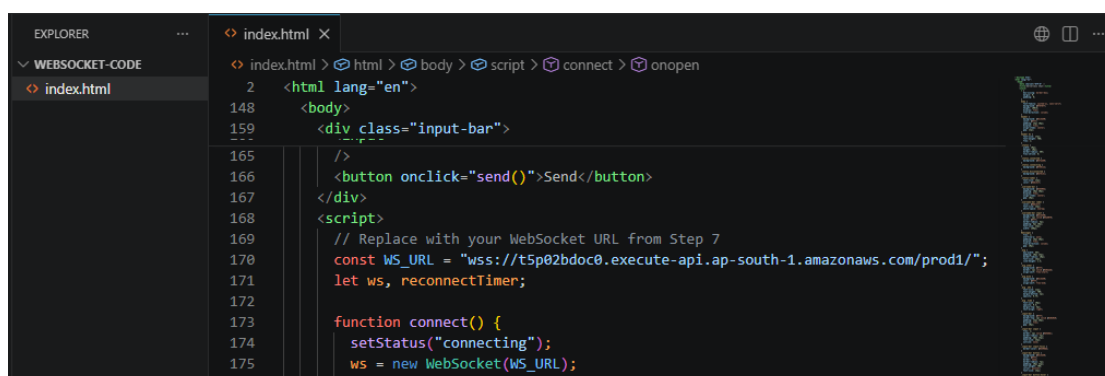
Step 12: Connect Frontend to WebSocket API

Connected the browser to the WebSocket endpoint.

Example:

wss://<api-id>.execute-api.ap-south-1.amazonaws.com/prod1

Purpose: Establishes a persistent WebSocket connection.

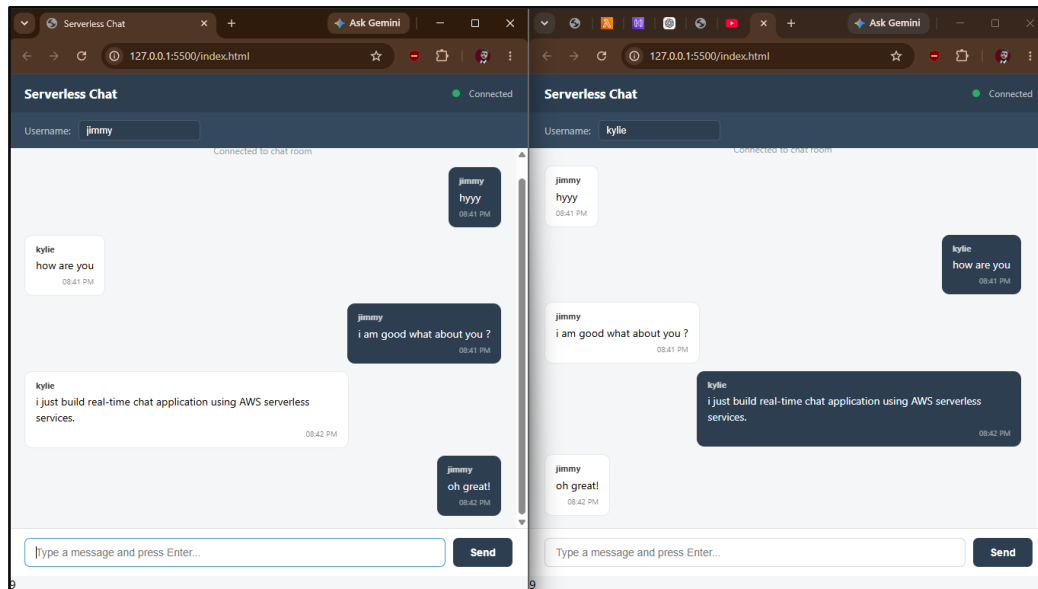


Step 13: Test Chat Application

Opened the application in multiple browser tabs.

Verified:

- Connection established
- Messages delivered instantly
- Disconnect handled correctly



Step 14: Monitor Logs

Used CloudWatch Logs.

Purpose: Debug Lambda execution and identify errors.

Timestamp	Message
2026-06-23T15:12:39.467Z	START RequestId: f8b32677-629e-477e-a605-51ee8a7d14fc Version: \$LATEST
2026-06-23T15:12:39.801Z	END RequestId: f8b32677-629e-477e-a605-51ee8a7d14fc
2026-06-23T15:12:39.802Z	REPORT RequestId: f8b32677-629e-477e-a605-51ee8a7d14fc Duration: 334.26 ms Billed Duration: 335 ms Memory Size: 128 MB Max Memory...
2026-06-23T15:12:46.250Z	START RequestId: 1786c4fd-e31f-47b8-9852-059c4fc9b431 Version: \$LATEST
	START RequestId: 1786c4fd-e31f-47b8-9852-059c4fc9b431 Version: \$LATEST
2026-06-23T15:12:46.561Z	END RequestId: 1786c4fd-e31f-47b8-9852-059c4fc9b431
	END RequestId: 1786c4fd-e31f-47b8-9852-059c4fc9b431
2026-06-23T15:12:46.562Z	REPORT RequestId: 1786c4fd-e31f-47b8-9852-059c4fc9b431 Duration: 310.64 ms Billed Duration: 311 ms Memory Size: 128 MB Max Memory...
	REPORT RequestId: 1786c4fd-e31f-47b8-9852-059c4fc9b431 Duration: 310.64 ms Billed Duration: 311 ms Memory Size: 128 MB Max Memory Used: 97 MB

No newer events at this moment. Auto retry paused. [Resume](#)

Activate Windows
Go to Settings to activate windows.

[Back to top](#)

Troubleshooting

Issue 1: Connection Failed

Solution

- Verify WebSocket endpoint.
- Deploy the API again.
- Check route integrations.

Issue 2: KeyError: WS_ENDPOINT

Solution

- Add the WS_ENDPOINT environment variable.
- Remove the wss:// protocol.
- Use the HTTPS Management API endpoint.

Conclusion

This project successfully demonstrates a **serverless real-time chat application** using AWS services. API Gateway manages WebSocket connections, Lambda processes events, and DynamoDB stores active connection IDs. The application provides instant communication without managing servers, making it scalable, cost-effective, and suitable for modern cloud-based real-time applications.